Homework 9

1 1. Recurrences

1.1 (a) A terminates when n is a non-negative `int`

Proof by Strong Induction

- Base case: $n = 0$
  - the function terminates immediately if $n = 0$ that return 0.
- Inductive step
  - every odd integer input $n > 1$ returns a recursive call to even input $2(n-1)$.
  - after dividing by 2 in even integer input case, $n - 1$.
  - So, each odd/even call reduces the non-negative integer input by 1.
  - after $2 \times n$ where n iterations where n is the initial non-negative integer input, the function will eventually return $n = 0$.

Therefore, A terminates when its input n is a non-negative integer.

1.2 (b) recurrence for the running time of A

$T(n) = T(n/2) + \Theta(1)$: even case $T(n) = T(2(n-1)) + \Theta(1)$: odd case

1.3 (c)

$$
\begin{aligned}
&T(n/2) + \Theta(1) \\
&a = 1, b = 2, d = 0 \\
&0 = log_2(1) \rightarrow T(n) = \Theta(n^0 logn) \\
&= \Theta(logn)
\end{aligned}
$$

$$
\begin{aligned}
&T(2(n-1)) + \Theta(1) \\
&= \Theta(1) + 2\Theta(1) + 4\Theta(1) + ... + 2^{n-1}\Theta(1) \\
&= 2^n\Theta(1) = \Theta(2^n)
\end{aligned}
$$

1.4 (d)

1. depth: $O(logn)$
2. the number of leaves: $O(1)$; exactly 1 leaf left of base case

1.5 (e)

It computes the number of 1s of the input n in binary expression.

- e.g. 10 = `0b1010`
  - $A(10)$ where n = 10 : 2

- $A(10) \rightarrow A(5) \rightarrow A(8) + 1$(this is $\Theta(1)$operation) $\rightarrow A(4) \rightarrow A(2) \rightarrow A(1) + 1 \rightarrow A(0)$(base case reached)
- count the number of 1s: $1 + 1 = 2$.

2 2. Carpets

2.1 1. Algorithm

2.1.1 plain English version

I/O

- Input: $n = 2^k$ (size of the room) and coordinate $(x, y) \in \{1, ..., n\} \times \{1, ..., n\}$ of the 1×1 covered square.
- Output: list of carpet pieces to fill up $2^k \times 2^k$ floor

Divide and conquer recursive call

1. divide the current square floor in 4 quadrants
2. we decide which quadrant we will recursively call by getting which quadrant the input 1×1 covered resides
3. we cover the floor of the other three quadrant without the input covered tile with the matching carpet piece.
4. as we cover the floor with the matching carpet pieces depending on the location of given 1×1 cover resides, we update the coordinate and its matching pieces in the list
5. we process the whole quadrants recursively with the initially added 1 cover for the other 3 quadrants with L-shaped cover piece at the middle. stop edge case for the recursion would be to reach 1×1 grid.
6. return the cover piece list.

2.1.2 Pseudocode

```
carpet(n: int, cx: int, cy: int):
    /*IO
    Input: n: size of the floor; power of 2
    cx: x coordianatce of the already-covered floor
    cy: y coordinate of the already-covered floor

    Output: list of tuples that contains: (x, y, L-shaped carpet piece)
    */

    // 1. base case: we stop when we don't have space to put L-shaped carpet piece
    if n = 1: // size of 1
        return empty list

    center = n / 2 // half the size so make 4 quadrants
```

```
quadlist = {
    TL = (center, center)
    TR = (center + 1, center)
    BL = (center, center + 1)
    BR = (center + 1, center + 1)
}
// find which quad does (cx,cy) reside
if cx <= center:
    if cy <= center:
        quad = TL
    else:
        quad = BL
else:
    if cy <= center:
        quad = TR
    else:
        quad = BR
// locating other three quads; need to process all four quadrants
opposite = {TL: BR, TR: BL, BL: TR, BR: TL} // already-covered: opposite direction to fillu
fillup = quadlist[opposite[quad]] // get the coordinates of the directional l-shape cover p

L_tiles = [(fillup.cx, fillup.cy, quad + "direction")] // output L tile

subgrid = {
    TL: (0,0,quadlist[TL]),
    TR: (center, 0, quadlist[TR]),
    BL: (0, center, quadlist[BL]),
    BR: (center,center, quadlist[BR])
}
// recursion order: first, quad that the initially given $1 \time 1$ cover resides
// then, do fillups at quadrants clockwise-neighbor
recurse_order = [quad] + [for i in subgrid: subgrid[i].key if subgrid[i].key == quad: conti

// divide subproblems by quad
for q' in recurse_order:
    (x_offset, y_offset, center_cell) = subgrid[q']

    // recurse until we reach the base case
    if q' = quad:
        sub_x = cx - x_offset
        sub_y = cy - y_offset
    else:
        sub_x = center_cell.cx - x_offset
        sub_y = center_cell.cy - y_offset
```

```
    // recurse on smaller subproblems
    sub_tiles = carpet(center, sub_x, sub_y)

    // append the l shape tiles in the output
    for (x, y, dir) in sub_tiles:
        L_tiles.append(x + x_offset, y + y_offset, dir)
return L_tiles
```

2.2 proof

To prove the algorithm `carpet(n, cx, cy)` is correct:

- for each recursive call:
  - we get a square subgrid(subfloor) with exactly one already-covered cell
  - return tiles that cover the other cells exactly once

  Given $n = 2^k$:

  1. base case: 1×1 subgrid has one already-covered cell that fills up the whole subgrid: return no tiles
  2. Inductive step: assume this holds as well for $n/2 \times n/2$ subgrids.
     - because the algorithm checks four $n/2 \times n/2$ subgrids to check which subgrid has the input already-covered cell and put exactly one 1×1 cover for every other subgrid, the claim for each recursive call holds.
     - for each $n/2^k \times n/2^k$ subgrids that has exactly one 1×1 cover, the leftover grid to fillup, considering we only have L-shaped covers, is $\frac{(n/2^k \times n/2^k)-1}{3}$. This is exactly the same with the number of carpet pieces that friend gave us. Therefore, there is no overlap for filling up the whole four subgrid.

  Therefore, the algorithm is always correct

2.3 recurrence for the worst running time in terms of n and the running time

$T(n) = 4T(n/2) + \Theta(n^2) : a = 4, b = 2, d = 2$

- we divide into 4 subproblems (quadrants)
- we make the problem size by half
- tile list conversion: n^2 (nested for loop)

Solve by Master method $d = 2 = log_2(4) \rightarrow T(n) = \Theta(n^2 logn)$